PYTHON (for beginners)

CHAPTER-1 (MODULES, COMMENTS & PIP

* .py = It is a extension · ex-like - new.py
like - andyy.pdf
new2.py

→ **Pip** - to install, Module download or It is package manager for python
like - pip install ---- , pip install pyjokes
pip install flask
code written by someone else (usually)

→ **Module** - It is a file containing code written by somebody else (usually) which can be imported & used in our programs

→ **Types of Module**

Two Types:

1. Built in Modules (Preinstalled in python) ex - os, random etc
2. External Modules (Need to install using pip)
for ex - Tensorflow, flask etc.

→ **Using Python as a calculator**

* Use python as a calculator by typing "python" + ↵ on the terminal.
* This opens REPL or Read Evaluate Print loop.

→ **Comments** - add a # at the start.
It is used to write something which the programmer does not want

1. **Single line comment** : for single line just add "#" at the start of line

ex - #This is a single line comment.

② **Multiline Comments**: to write multiline, use "#" at each line or you can use multiline string (""" """)

ex-

```
""" This is an amazing
example of a Multiline
Comment! """
```

Use (''' ''') to print long line

↳ like - print('''
_ _ _ _
_ _ _ _
''')

Practice set

1. Write a program to print Twinkle twinkle in python.
2. Use REPL and print the table of 5 using it.
3. Install external Module & use it to perform an operation of your interest.
4. Write a python program to print the contents of a directory using the OS Module. search online for the function which does that.
5. Label the program written in problem 4 with comments.

CHAPTER-2 (VARIABLES & DATABASE

→ **Variable** - It is the name given to memory location in a program.

for ex-

```
a = 30          #variable = container to store a value.
b = "harry"     # keywords = reserved words in python.
c = 91.22       # Identifiers = class/function/variable name
```

→ **Data Type**

following Types

1. Integers
2. floating Point numbers
3. strings
4. Booleans
5. None

Python is a fantastic language that automatically identifies the type of data for us.

ex

```
a = 11          # identifies a as class <int>
b = 88.24       # identifies b as class <float>
name = "harry"  # identifies name as class <str>
```

→ Rules for defining a variable name (Also applies to other identifiers)

- A variable name can contain alphabets, digits & Underscores.
- A variable name can only start with an alphabet & Underscores.
- A variable name can't start with a digit.
- No white space is allowed to be used inside a variable name.

ex - harry, one8, seven, _seven etc.

→ operators in python

following are some common operator in python.

1. Arithmetic operators: +, -, *, / etc.
2. Assignment operators: =, +=, -= etc.
3. Comparison operators: ==, >, >=, <, != etc.
4. Logical operators: and, or, not.

→ Type() Function and Typecasting

Type() function is used to find the data type of a given variable in python.

ex

```
a=31
type(a)   #class <int>

b="31"
type(b)   #class <str>
```

- A number can be converted into a string & vice versa (if possible)

- There are many functions to convert one data type into another.

```
str(31)    => "31"   # Integer to string conversion
int("32")  => 32     # string to integer conversion
float(32)  => 32.0   # integer to float conversion
```

... and so on

Here "31" is a string literal and 31 a numeric literal.

→ Input() Function

This function allows the user to take input from the keyboard as a sitring.

```
A = input("enter name")  # if a is "harry", user
                           entered harry
```

- It is important to note that the output of input is always a string (even is a number is entered).

Comparism operator only gives T/F. mean only boolean

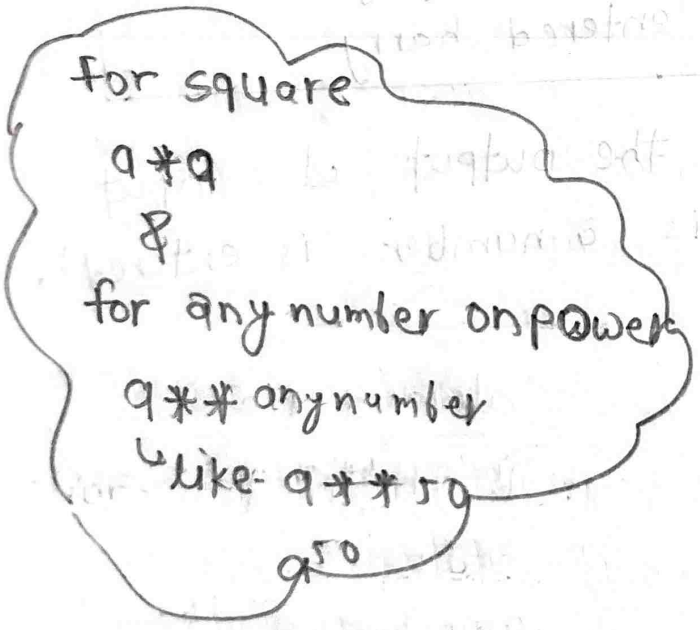
always use == to check two values are. equal or not

alt + shift + ↓
↳ for replicate line
ex- ③ Hello
alt + shift + ↓
↳ ④ Hello

for remainder find a % b – divide

Practise set

1. Write a python program to add two numbers.
2. Write a python program to find remainder when a number is divided by z.
3. check the type of variable assigned using () function.
4. Use comparison operator to find out whether a given variable a is greator than 'b' or not.
   Take a=34 and b=80
5. Write a python to find an average of two numbers entered by the user.
6. Write a Python program to calculate the equare of a number entered by the user.



CHAPTER-3 (STRING)

→ **Strings** :- It is a data type in python. It is sequence of characters enclosed in quotes.

- We can primarily write a string in these three ways.

```
a = 'harry'        #single quoted string
b = "harry"        #Double quoted string
c = '''harry'''    #Triple quoted string
```

→ **String slicing**

A string in python can be sliced for getting a part of the strings.

- Consider the following string

Name = H a r r y ⇒ length = 5

0	1	2	3	4
(-5)	(-4)	(-3)	(-2)	(-1)

we can also number them in negative. Just number them in opposite.

The index in a string starts from 0 to (Length - 1) in Python. In order to slice a string, we can use the following syntax:

```
Sl = name [ind_start : ind_end]
```

ind_start → first index included

ind_end → last index is not included

S[0:3] returns "har" → characters from 0 to 3

S[1:3] returns "ar" → characters from 1 to 3

- **Negative indices**: Negative indices can also be used as shown in the figure above. -1 corresponds to the (length-1) index, -2 to (length-2)

→ Slicing with Skip Value

We can provide a skip value as a part of our slice like this.

```
word = "amazing"
word [1:6:2]  # "mzn"
```

Other advanced slicing techniques:

```
word = "amazing"
word = [:7]    # word [0:7] - 'amazing'
word = [0:]    # word [0:7] - amazing
```

→ String Functions

some of the mostly used functions to perform operations on or manipulate string size.

1. **len() function** - Used for returns the length of the strings.

```
len("Harry")  #returns 5
```

2. **string.endswith("rry")** - This function tells whether the variable string ends with the string "rry" or not. If string is "harry", it returns true for "rry" since Harry ends with rry.

```
MyString = harry
MyString.endswith ("rry")
```

③ string.count("c") - counts the total number of occurances of any characters.

```
string = "abracadobra"
count = string.count("c")
print (count)    # Output : 1
```

④ string.capitalize() - This function capitalize the first character of a given string.

```
S = "hello world"
capitalized_string = S.capitalize()
print (capitalized_string)    # output : "Hello world"
```

⑤ string.find(word) - This function friends a word & returns the index of first occurrence of that word in the string.

```
S = "Hello world"
index = s.find ("world")
print (index)    # Output: 6
```

⑥ string.replace (old word, new word) - This function replace the old word with new word in the entire string.

```
S = "hello world"
replaced_string = s.replace ("world", "Python")
print (replaced_string)    # Output : "Hello Python"
```

Escape sequence characters

Sequence of characters after backslash "\" → Escape sequence character

- Escape sequence character comprise of more than one character but represent one character when used within the string

example - \n , \t , \' , \\ etc

\n → Newline
\t → tab
\' → single quote
\\ → backslash

for Line
Use Alt + Shift + ↑
↳ to move up
Use Alt + Shift + ↓
↳ to move down

Use f string for directly shown the req name like Practice set-1

ex
```
name = input ("Enter Your name")
print (f" Good Morning {name} ")
```

Practice set 1

1. Write a python program to display a user entered name followed by Good Afternoon using input() function.
2. Write a program to fill in a letter template given below with name & data.

```
letter= '''

      Dear <|Name|>,
      You are selected!
       <|Date|>
       '''
```

3. Write a program to detect double space in a string.
4. Replace the double space from problem 3 with single spaces.
5. Write a program to format the following letter using escape sequence characters.

CHAPTER-4 (LISTS & TRUPLES)

- Python lists are containers to store a set of value of any data type.

```
friends = ["apple", "akash", "roban", 7, false]
```

str() — can store value of any data type — int() — bool()

→ List Indexing

A list can be indexed just like a string.

```
L1 = [7, 9, "harry"]

L1[0]   #7
L1[1]   #9
L1[70]  #error
L1[0:2]   #[7, 9]   #list slicing
```

→ List Methods

Consider the following list:

ex-

```
L1 = [1, 8, 7, 2, 21, 15]
```

- L1.sort(): updates the list to [1, 2, 7, 8, 15, 21]
- L1.reverse(): updates the list to [15, 21, 2, 7, 8, 1]
- L1.append(8): adds 8 at the end of the list
- L1.insert(3, 8): This will add 8 at 3 index
- L1.pop(2): will delete element at index 2 & return its value
- L1.remove(21): will remove 21 from the list.

→ Tuples in python

A Tuples is an immutable (non delete) data type in Python.

```
a = ()      # empty tuple
a = (1, )   # tuple with only one element need a comma
a = (1,7,2) # tuple with more than one element
```

→ Tuple Methods

Consider the following Tuple : 1. count()
2. index()

* ex -

```
a = (1,7,2)
```

① a.count(1) : a count(1) will return number of times 1 occurs in a

② a.index(1) will return the index of first occurance of 1 in a

Other Useful Built-in Functions of Tuple

- len(t) - numbers of element
- max(t) - largest number
- min(t) - smallest element
- sum(t) - sum of element
- sorted(t) - returns a sorted list

*t is variable mean it can be anything you won't

Note- Tuples are immutable, so Methods like append(), remove(), etc, do not exist

Practice set

1. Write a program to store seven fruits in a list entered by the user.
2. Write a program to accept marks of 6 students & display them in a sorted manner.
3. check that a type cannot be changed in python.
4. Write a program to sum a list with 4 numbers.
5. Write a program to count the number of zeros in the following tuple:

```
a = (7, 0, 8, 0, 0, 9)
```

to Autocorrect in terminal
write first two
then TAB
but same name other
not exist

CHAPTER-5 (DICTIONARY & SETS)

→ **Dictionary** - It is a collection of keys-value pairs.

syntax :

```
a = {"key" : "value" ,
     "harry" : "code" ,
      "marks" : "100"
      "list" : [1, 2, 9]
    }
a ["key"]    # prints "value"
a [list]     # prints [1, 2, 9]
```

→ **Properties of Python dictionaries**

1. It is unordered.
2. It is mutable
3. It is indexed.
4. cannot contain duplicate keys.

→ **Dictionary Methods**

Consider the following dictionary.

```
a = {"name" : 'harry"
     "from" : "india"
     "marks" : [92, 98, 96]
```

- a.items() : Returns a list of (key, value) tuples.
- a.keys() : Returns a list containing dictionary's keys.

- a.update({"friends":4}) : Updates the dictionary with supplied key values pairs.
- a.get("name"): Returns the value of the specified keys (and value is returned eg. "harry" is returned here).

→ Sets in Python

Set is a collection of non-repetitive elements.

```
S = set()          # no repetition allowed
s.add(1)
S.add(2)           # or set = {1,2}
```

→ Properities of sets

1. Sets are unordered.
2. Sets are unindexed
3. There is no way to change items in sets.
4. sets cannot contain duplicate value.

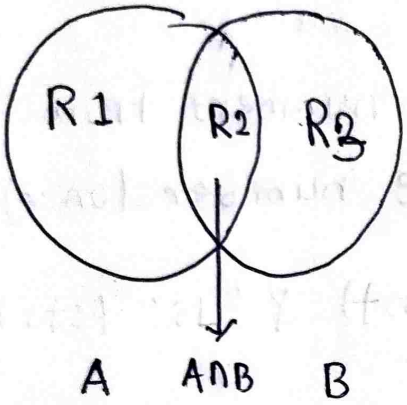
→ operations on sets

Consider the following set:

```
S = {1,8,2,3}
```

- len(s): Returns 4, the length of the set
- s.remove(8): Updates the set s & remove 8 from s.
- s.pop(): Removes an arbitary element from the set & return the element removed.
- s.clear(): empties t set s.

- S.union({8, 11}) : Returns a new set with all items from both sets. {1, 8, 2, 3, 11}
- S.intersection ({8, 11}) : Return a set which contains only item in both sets {8}.



$R2 = A \cap B$

$R1 + R2 + R3 = A \cup B$

$R1 + R3 \Rightarrow A \Delta B$

$R1 \Rightarrow A - B$

$R3 \Rightarrow B - A$

Practice set

1. Write a program to create a dictionary of Hindi words with values as their English translation. Provide user with an option to look it up!

2. Write a program to input eight numbers from the user & display all the unique numbers(once).

3. Can we have a set with 18 (int) & "18" (str) as a value in it? yes

4. What will be the length of following sets:

```
s = set()
s.add(20)
s.add(20.0)
s.add('20')  # length of s after these operations
```

Length is 2
Because python properties is he compare value.
so 20 == 20.0

5. s = {}
What is the type of s? → dict

6. Create an empty dictionary. Allow 4 friends to enter their favourite language as value & use key as their names. Assume that the names are unique.

7. If the names of 2 friends are same; what will happen to the program in problem 6?

Values get updated

(8) If language of two friends are same: what will happen to the program in problem 6?

↳ Nothing will happen

(9) can you change the values inside a list which is contained in set S; S = {8, 7, 12, "Harry", [1,2]}

↳ In set indexing not happen
↳ we can't include list in a set
↳ because sets in python require all their elements to be immutable & hasable

CHAPTER-6 (CONDITIONAL EXPRESSION)

→ **Conditions** :- some desicions depends on condition like- Sometimes we order Ice-cream online if the day sunny. etc.

In Python, we must be able to execute instructions on a condition(s) being met. This is what conditionals are for.

→ **If else & elif in python**

If "else & Elif" statement are a multiway decision taken by our program due to certain conditions in our code.

Syntax :

```
if (condition1) :  # if condition1 is True
    print ("yes")

else (condition2) : # if condition2 is True
    print ("no")

else :                          # otherwise
    print ("maybe")
```

- **Code example:**

```
a=22
if (a>9)
    print ("greater")

else :
    print ("lesser")
```

→ **Relationship operators** : It is used to evaluate conditions inside the if statement. some examples of relational operators are:

== : equals

>= : greater than /equal to

<= : lesser than /equal to

→ **Logical operators** : these operators operates on conditional statements

example:

- and - true if both operands are true else false.
- or - true if at least one operands is true or else false.
- not - inverts true to false & false to true.

→ **ELIF clause** : "Elif" in python means [else if] An if statement can be chainged together with a lot of these Elif statement followed by an else statement:

```
if (condition1):
   #code
elif (condition2):  # this ladder will stop once a condition
                      in an if or elif is met.
   #code
elif (condition3):
   # code
else :
   # code
```

Important Note

1. There can be any number of Elif statements.
2. Last else is executed only if all the conditions inside Elif fail.

the space after (if, else, elif) is called "Indent"

Apne variable ke name always be descriptive mean detailed full name

Practice Set

1. Write a program to find the greatest of four numbers entered by user.

2. Write a program to find out whether a student has passed or failed if it requires a total of 40% and at least 33% in each subject to pass. Assume 3 subjects and take marks as an input from the user.

3. A spam comment is defined as a text containing following keywords:
"Make a lot of money", "buy now", "subscribe this", "click this". Write a program to detect these spams.

4. Write a program to find whether a given username contain less than 10 character or not.

5. Write a program which finds out whether a given name is present in a list or not.

6. Write a program to calculate the grade of a student from his marks from the following scheme:

90 - 100 => EX
80 - 90 => A
70 - 80 => B
60 - 70 => C
50 - 60 => D
< 50 => F

(7) Write a program to find out whether a given post is talking about "Harry" or not.

CHAPTER -7 (LOOPS IN PYTHON)

→ **Loops** : Loops make it easy for programmer to tell the computer which set of instructions to repeat & how!

→ **Types of loops in python**

Primarily there are two types of loops in python.

- While loops
- for loops

→ **While loop**

Syntax:

```
while (condition) :   # The block keeps executing until the condition is true
    #Body of the loop
```

In while loops, the condition is checked first. If it evaluates to true, the body of loop is executed otherwise not!

In the loop is entered, the process of [condition check & execution] is continued until the condition becomes False.

ex- write a program to print 1 to 50 using a while loop

Sol-

```
i = 0
while i < 5:                    # Print "Harry" - 5 times!
    print ("Harry")
    i = i + 1
```

→ **For loop** : for loop is used to iterate through a sequence like list, tuple or string [iterables]

Syntax:

```
l = [1, 7, 8]
for item in l:
    print(item)  #prints 1, 7 & 8
```

→ **Range Function in python**

The range [ ] function in python is used to generate a sequence of number.

We can also specify the start, stop & step size as follows:

```
range(start, stop, step_size)
#step size is usually not used with range()
```

→ **An Example demonstrating Range ( ) Function**

```
for j in range(0,7) : #range(7) can also be used
    print(j) # Print 0 to 6
```

→ **For loop with else**

An optional else can be used with a for loop if the code is to be executed when the loops exhausts.

example:

```
l = [1,7,8]
for item in l:
    print(item)

else:
    print("done")   #this is printed when the loop
                      exhausts!
```

Output:

```
1
7
8
Done
```

→ The Break statement

"Break" is used to come out of the loop when encountered. It instructs the program to exit the loop now.

example:

```
for i in range(0,80):
    print(i)        # this will print 0,1,2 &3
    if i==3
        break
```

→ **Continue statement** : It is used to skip the current iteration of the loop & continue with the next one. It instructs the Program to "skip this iteration".

example:

```
for i in range(4):
    print ("printing.")
    if i==2    #if i is 2, the iteration is skipped
        continue
    print(i)
```

→ **Pass statement** : Pass is a null statement in python. It instruct to "do nothing".

example:

```
i = [1, 7, 8]
for item in i:
    pass          #without pass, the program will
                    throw an error
```

In string is use to fit variable

Practice set

1. Write a program to print multiplication table of a given number using for loop.

2. Write a program to greet all the person names stored in a list "l" and which start with s.

```
l = ["Harry", "soham", "sachin", "Rahul"]
```

3. Attempt Problem 1 using while loop.

4. Write a program to find whether a given number is prime or not.

5. Write a program to find the sum of first n natural numbers using while loop.

6. Write a program to calculate the factorial of a given number using for loop.

7. Write a program to print the following star pattern.

```
  *
* * *
* * * * *   for n = 3
```

8. Write a program to print the following star pattern:

```
  *
* *
* * *
```

(9) Write a program to print the following star pattern.

```
* * *
*   *    for n=3
* * *
```

(10) Write a program to print multiplication table of n using for loops in reversed order.

CHAPTER: 8 (FUNCTION & RECURSIONS)

→ **Function** :- A function is a group of statements task.

When a program gets bigger in size & its complexity grows, it gets difficult for a program to keep track on which place of code is doing what!

A function can be reused by the programmer in a given program any number of

→ **Example & syntax of a function**

The Syntax of a function looks as follows:

```
def fun1():
    print("hello")
```

This function can be called any number of times, anywhere in the program.

→ **Function Call**

Whenever we went to call a function, we put the name of the function followed by parentheses as follows:

```
fun1()  #This is called function call.
```

→ **Function Definition** : The part containing exact set of instructions which are executed during the function call.

→ **Types of functions in Python**

two types of functions in Python:

- Built in functions (Already present in python)
- User defined functions (Defined by the user)

examples of built in functions includes len(), print(), range() etc.

The func1() function we defined is an example of user defined function.

→ **Functions with Arguments** :

A function can accept some value it can work with. We can put these values in the parentheses.

A function can also return value as shown below:

```
def greet(name):
    gr = "hello" + name
    return gr

a = greet("harry")
# a will now contain "hello harry"
```

→ Default Parameter value

We can have a value as default as default argument in a function.

If we specify `name = "stranger"` in the line containing def, this value is used when no argument is passed.

Example:

```
def greet(name = "stranger"):
    #function body
greet() #name will be "stranger" in function body (default)
greet("harry") #name will be "harry" in function body (passed)
```

→ **Recursion**: It is a function which calls itself.

It is used to directly use a mathematical formula as function.

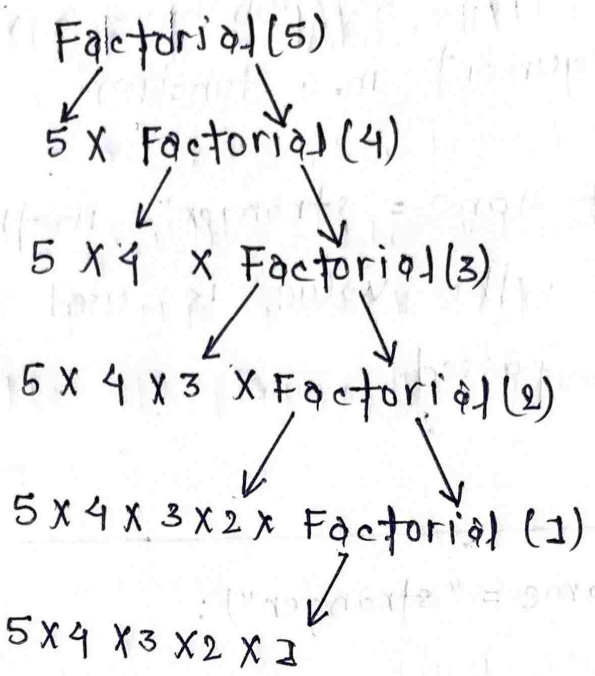
example:

$$factorial(n) = n \times factorial(n-1)$$

This function can defined as follows:

```
def factorial(n)
    if i==0 or i==1:   #Base condition which doesn't call the function any futher
    return 1
else:
    return n*factorial(n-1) #function calling itself
```

This works as follows:



The programmer needs to be extremely careful while working with recursion to ensure that the function doesn't infinitely keep calling itself. Recursion is sometimes the most direct way to code on algorithm.

$\frac{c}{5} = \frac{(f-32)}{9}$

c = celcius
f = fahrenheit

write end = "\n"

to avoid new line to the end

after return
after code never calls the code end there

Practice Set

1. Write a program using functions to find greatest of three numbers.
2. Write a python program using function to convert celcius to Fahrenheit.
3. How do you prevent a python print() function to print a new line at the end.
4. Write a recursive function to calculate the sum of first n natural numbers.
5. Write a python function to print first n lines of the following pattern:

```
* * *
* *
*
```

- for n=3

6. Write a python function which converts inches to cms.
7. Write a python function to remove a given word from a list ad strip it at the same time.
8. write a python function to print multiplication table of a given number.

PROJECT : 1 (SNAKE, WATER, GUN GAME)

Due- PROJECT - We all have played snake, water gun game in our childhood. If you haven't, google the rules of this game & write a python program capable of playing this game with the user.

for choose random & import random
use random.choice
like : computer = random.choice([-1, 1, 0])

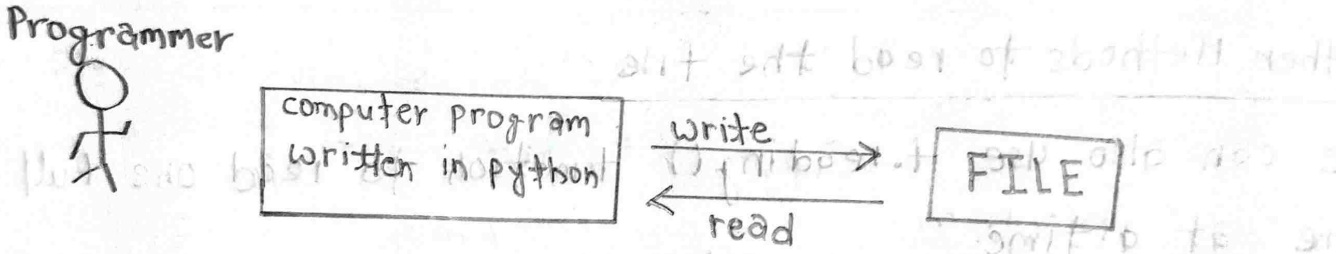
you can make if, elif ladder in else also

for change anything which are repeated in lines.
(i) Right click
(ii) change all occurances & edit

CHAPTER: 9 (FILE I/O)

"RAM" is volatile, & all its contents are lost once a program terminates in order to persist the data forever, we use files.

A file is data stored in a storage device. A Python program can talk to the file by reading content from it & writing content to it.



* RAM = volatile
* HDD = Non volatile

→ Types of files

There are 2 types of files:

1. Text files (.txt, .c, etc)
2. Binary files (.JPG, .dat, etc)

→ opening a file

Python has an open() function for opening files. It takes 2 parameter: filename & mode

```
#open("filename", "mode of opening (read mode by default)")
open("this.txt", "r"
```

→ Reading a file in python

```
# Open the file in read mode
f = open("this.txt", "r")
# Read its contents
text = f.read()
# Print its contents
print(text)
# close the file
f.close()
```

→ Other Methods to read the file

We can also use f.reading() function to read one full line at a time.

```
f.readline()        # Read one line from the file
```

→ Modes of opening a file

- r = open for reading
- w = open for writing
- a = open for appending
- + = open for updating
- 'rb' will open for read in binary mode.
- 'rt' will open for read in text mode

→ Write files in Python

In order to write to a file, we first open it in write or append mode after which, we use the python's f.write() method to write to the file

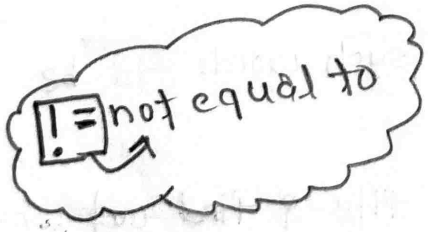
```
#Open the file in write mode
f = open("this.txt", "w")
#write a string to the file
f.write("this is nice")
#close the file
f.close()
```

→ With statement

The best way to open & close the file automatically is the with statement.

```
#open the file in read mode using 'with', which
  automatically closes the file
with open("this.txt", "r") as f:
        #Read the contents of the file
        text = f.read()

#Print the contents
print(text)
```



Practice Set

1. Write a program to read the text from a given file "poem.txt" & find out whether it contains the word "twinkle".

2. The game() function in a program lets a user play a game & returns the score as an integer. You need to read a file "Hi-score.txt" which is either blank or contains the previous Hi-score. You need to write a program to update the Hi-score whenever the game() function breaks the Hi-score.

3. Write a program to generate multiplication table from 2 to 20 & write it to the different files. Place these files in a folder for a 13-year old.

4. A file contains a word "Donkey" multiple times. You need to write a program which replace this word with ###### by updating the same file.

5. Repeat program 4 for a list of such words to be censored.

6. write a program to mine a log file & find out whether it contains "python".

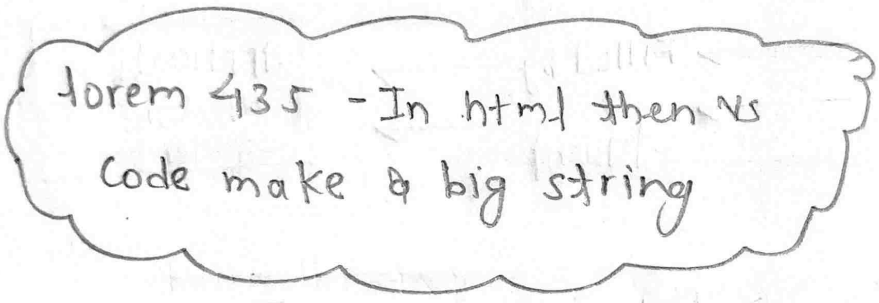
7. write a program to find out the line number where python is python is present from Que 6.

10. write a program to wipe out the content of file using python.

(8.) Write a program to make a copy of a a text file "this.txt".

(9.) Write a program to find out whether a file is identical & matches the content of another file.

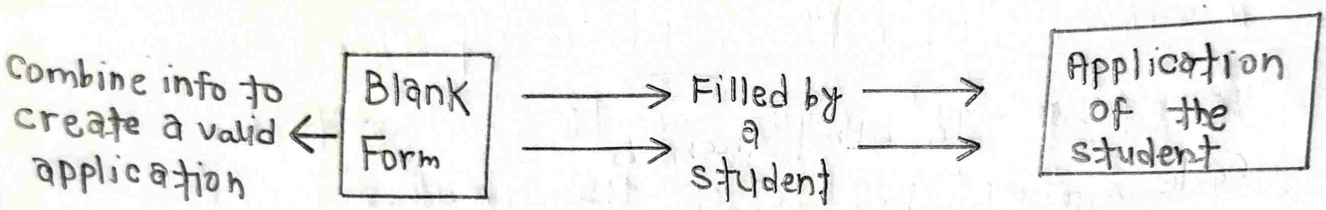
(11) Write a python program to rename a file to "renamed_by_python.txt

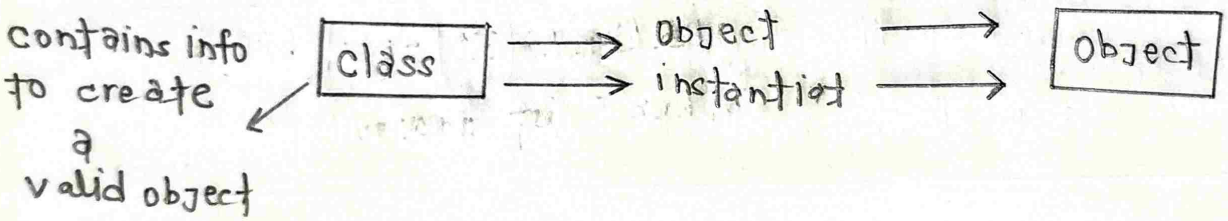


CHAPTER : 10 (OBJECT ORIENTED PROGRAMMING)

→ **Object oriented programming** - solving a problem by creating **object** in programming is called "object oriented programming".

→ **class** : It is a **blueprint** for creating object.





Syntax:

```
class Employee : #class name is written in Pascal scale
  # Methods & variables
```

→ **object**: An object is an **instantiation of a class**. When class is defined, a template (info) is defined. Memory is allocated only after installation.

Object of a given class can invoke the Methods available to it without revealing the implementation detailed to the user. — **Abstractions & Encapsulation**

→ Modelling a problem in loops

We identify the following in our problem.

- Noun → class → Employee
- Adjective → Attributes → name, age, salary
- Verbs → Methods → getsalary(), increment()

→ class Attributes

An attributes that belongs to the class rather than a particular object.

example:

```
class Employee:
    company = "Google" #specific to Each class
harry = Employee()  #object Instantiation
harry.company
Employee.company = "youtube" # changing class
                               Attribute
```

→ Instance attributes

An attribute that belongs to the Instance (object). Assuming the class from the previous example:

```
harry.name = "harry"
harry.salary = "30k"  #Adding instance attribute
```

Note: Instance attributes, take preference over class attributes during assignment & retrieval.

When looking up for harry.attribute it checks for the following.

1) Is attributes present in object?
2) Is attributes present in class?

→ Self parameter

In self parameter, self refers to the instance of the class. It is automatically passed with a function call from an object.

```
harry.getSalary() #here self is harry
# equivalent to Employee.getsalary(harry)
```

The function getsalary() is defined as:

```
class Employee:
    company = "Google"
    def getsalary(self):
        print("salary is not there")
```

→ Static Method

Sometimes we need a function that does not use the self-parameter. We can define a static Method like this.

```
@staticmethod      #decorator to mark greet as a
def greet():          static Method
    print("Hello user")
```

→ __INIT__() constructor

- __init__() is a special method which is first run as soon as the object is created
- __init__() method is also known as constructor. It takes self-argument & can also take further argument.

for example:

After there is a space
def __init__

```
class Employee:
    def __init__(self,name):
        self.name = name

    def getsalary(self):
        ...

harry = Employee("Harry")
```

__ = double underscore is called dunder Method

Practice Set

1. Create a class "Programmer" for storing information of few programmers working at Microsoft.
2. Write a class "calculator" capable of finding square, cube & square root of a number
3. Create a class with a class attributes a : create an object from it & set 'a' directly using object.a=0. Does this change the class attributes.
4. Add a static method in problem 2, to greet the user with hello.
5. Write a class Train which has methods to book a ticket, get status (no of seats) & get fare information of train running under Indian railway.
6. Can you change the self parameter inside a class to something else (say "harry"). Try changing self to "slf" or "harry" & see the effects.

If we use slf instead of self there is no change

using changing occurances option

or we write any thing like - Andyy there will be NO change

CHAPTER:11 (INHERITANCE & MORE ON OOPS)

→ **Inheritance**:- It is a way of creating a new class form an existing class.

Syntax:

```
class Employee:    #Base class
    #code
class Programmer(Employee):  #Derived or child class
    #code
```

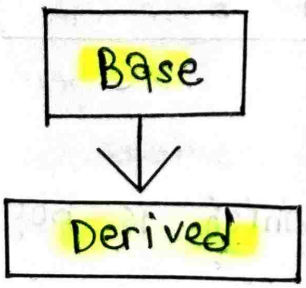
We can use the method & attributes of "Employee" in "Programmer" object.

Also, we can overwrite or add new attributes & methods in "Programmer" class.

→ **Types of inheritance**

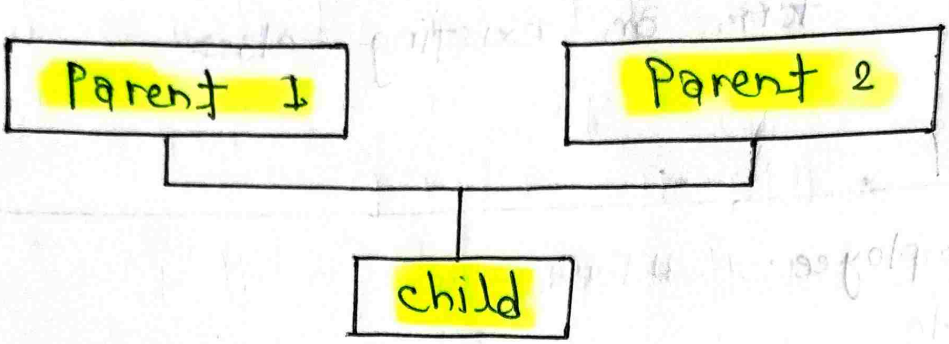
- single inheritance
- Multiple inheritance
- Multilevel inheritance

→ **single inheritance** :- It occurs when child inherits only a single parents class.



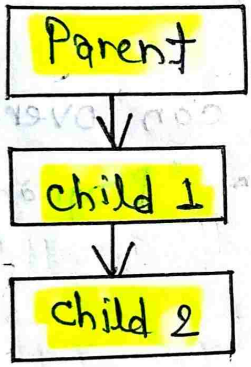
→ Multiple inheritance

It occurs when the child class inherits from more than one parent classes.



→ Multilevel inheritance

When a child class becomes a parent for another child class.



→ super() Method

It is used to access the methods of a super class in the derived class

```
super().__init__()
# __init__() calls contructor of the base class
```

→ class Methods :-

It is a method which is bound to the class & not the object of the class.

@classmethod decorator is used to create class method.

Syntax to create a class method:

```
@classmethod
def (cls, p1, p2):
    - - -
```

→ @property decorators

Consider the following class

```
class Employee:
    @property
    def name (self):
        return self.ename
```

If e=Employee() is an object of class employee., we can print(e.name) to print ename by internally calling name() function.

→ @.Getters & @.setters

The Methods name with "@property" decorator is called getter-method. We can define a function + @name.setter decorator like below:

```
@name.setter
def name (self, value):
    self.ename =value
```

→ Operator overloading in python

Operator in python can be overloaded using dunder methods.

These methods are called when a given operator is used on the objects.

operators in python can be overloaded using these following methods:

```
P1 + P2   # P1.__add__(P2)
P1 - P2   # P1.__sub__(P2)
P1 * P2   # P1.__mul__(P2)
P1 / P2   # P1.__truediv__(P2)
P1 // P2  # P1.__floordiv__(P2)
```

other dunder/magic methods in python

```
str__() # Used to set what gets displayed
          upon calling str(obj)
```

use "pass" to
write more code but the
if you won't

Practice set

1. create a class (2-D vector) & use it to create another class representing a 3-D vector.

2. create a class "Pets" from a class "Animals", & further create a class "Dog" from "Pets". Add a method "bark" to class "Dog".

3. create a class "Employee" & add salary & increment properties to it.
   write a method "salaryAfterIncrement" method with a @property decorator with a setter which changes the values of increment based on the salary.

4. Write a class "complex" to represent complex numbers, along with overloaded operators "+" & "*" which adds & multiplies them.

5. Write a class vector representing a vector of n dimensions. overload the + & * operator which calculates the sum & the dot(.) product of them.

6. Write __str__() methods to print the vectors as follows

$$7\hat{i} + 8\hat{j} + 10\hat{k}$$

Assume vector of dimension 3 for this problem.

(7) Override the __len__() method on vector of problem 5 to display the dimension of the vector.

CHAPTER :12 (ADVANCED PYTHON 1)

following are some of the newly added features in Python programming language.

→ Walrus operator

The walrus operator (:-) , introduced in python 3.8 , allows you to assign values to variables a part of an expression . This operator, named for its resemblance to the eyes & tusks of a walrus , is officially called the "assignment expression".

```
#Using walrus operator
if (n :- len([1, 2, 3, 4, 5])) > 3 ;
    print (f"List is too long ({n} elements , expected <=3)")
#output: List is too long (5 elements , expected <= 3)
```

In this example , n is assigned the value of len([1,2,3,4,5]) & then used in comparison within the if statement.

→ Types Definitions in Python

Type hints are added using the colon (:) syntax for variables & the -> syntax for function return types.

```
# Variable type hint
age : int = 25
```

```
# Function type hints
def greeting(name: str) -> str:
    return f"Hello, {name}!"
```

```
# Usage
print(greeting("Alice"))  # output: Hello, Alice!
```

→ Advance type hints

Python's typing module provides more advanced type hints, such as List, Tuple, Dict & Union.

You can import List, Tuple & Dict types from the typing module like this:

```
from typing import List, Tuple, Dict, Union
```

Syntax:

```
from typing import List, Tuple, Dict, Union
# List of Integers
numbers : List[int] = [1, 2, 3, 4, 5]
# Tuple of a string & an integer
person : Tuple[str, int] = ("Alice", 30)
# Dictionary with string keys & integer values
scores : Dict[str, int] = ("Alice: 90, "Bob": 85)
```

```
# Union type for variables that can hold multiple types
identifier : Union [int, str] = "ID123"
identifier = 12345    # Also valid
```

These annotations help in making the code self-documenting & allow developers to understand the data structure used at a glance.

→ Match case

Python 3.10 introduced the match statement, which is similar to the switch statement.

The basic syntax of the match statement involves matching a variables against several cases using the case keyword.

```
def http_status(status):
    match status
        case 200:
            return "OK"
        case 404:
            return "Not Found"
        case 500:
            return "Internal server Error
        case _:
            return "Unknown status"
```

```
#Usage
print(http_status(200))   #output : OK
print(http_status(404))   # output : Not Found
print(http_status(500))   # output : Internal server
                                Error
print(http_status(403))   #output : Unknown status.
```

→ **Dictionary merge & update operators**

New operator | and |= allow for **merging** & **updating** dictionaries.

```
dict1= {'a' : 1 , 'b' : 2}
dict2 = {'b' : 3 , 'c' : 4}
merged = dict1 | dict2
print(merged )   # output : {'a' : 1, 'b' : 3, 'c' : 4}
```

You can now use multiple context managers in a single **with** statement more clearly using the parenthesised context manager.

```
with (
      open('file1.txt') as f1,
      open('file2.txt') as f2
):
# Process files
```

→ Exception Handling in python

There are many built-in exceptions which are raised in python when something goes wrong.

Exception in python can be handled using a try statement. The code that handles the exception is written in the except clause.

```
try:
    #code which might throw exception
except Exception as e:
    print(e)
```

When the exception is handled, the code flow continue without program interruption.

We can also specify the exception to catch like below:

```
try:
    #code
except ZeroDivisionError:
    #code
except TypeError:
    #code
except:
    #code          #All other exceptions are handled
                    here.
```

→ Raising Exceptions

We can raise custom exceptions using the 'raise' keyword in python.

→ Try with else clause

sometimes we want to run a piece of code when try was successful.

```
try:
    #somecode
except:
    #somecode
else:
    #code          #This is executed only if
                    the try was succesful.
```

→ If __name__ = '__Main__' in python

'__name__' evaluates to the name of the module in python from where the program is ran.

If the module is being run directly from the command line, the '__name__' is set to string "__main__". Thus, this behaviour is used to check whether the module is run directly or imported to another file.

→ The global keyword used to Modify the variable outside of the current scope.

→ Enumerate function in python

this 'enumerate' function adds counter to an iterable & returns it

```
for i, items in list1:
    print(i, item) # prints the items of list 1
                   with index
```

→ List Comprehensions

List comprehensions is an elegent way to create lists based on existing lists.

```
list1 = [1, 7, 12, 11, 22,]
list2 = [i for item in list 1 if item >8 ]
```

Practice set

1. Write a program to open three files 1.txt, 2.txt & 3.txt if any these files are not present, a message without existing the program must be printed prompting the same.

2. Write a program to print third, fifth & seventh element from a list using enumerate function.

3. Write a list comprehension to print a list which contains the multiplication of a user entered number.

4. Write a program to display a/b where a & b are integers. if b=0, display infinite by handling the 'zeroDivisionError'.

5. Store the multiplication tables generated in problem 3 in a file named Tables.txt.

CHAPTER : 13 (ADV PYTHON II)

→ Virtual Enviroment

An enviroment which is same as the system interpreter but is isolated from the other python enviroment on the system

↳ Installation

to use virtual enviroment, we write :

```
pip install virtualenv    #Install the package
```

we create a new enviroment using:

```
virtualenv myprojectenv    #create a new venv
```

The next step after creating the virtual enviroment is to activate it.

we can now use this virtual enviroment as a seperate python installation.

→ PIP freeze command

"PIP freeze" returns all the package installed in a given python enviroment along with the versions.

```
pip freeze > requirements.txt
```

The above command create a file named "requirements.txt" in the same directory containing the output of "pip freeze".

We can distribute this file to other users, & they can recreate the same enviroment using:

```
pip install -r requirements.txt
```

→ Lambda Functions

function created using an expression using "Lambda" keyword.

syntax:

```
lambda argument : expressions
# can be used as a normal function
```

example:

```
square = lambda x: x*x
square(6)    # returns 36
sum = lambda a, b, c: a+b+c
sum(1,2,3)   #returns 6
```

→ Join method (strings)

creates a string from iterable objects.

```
l = ["apple" , "mango", "banana"]
result = ", and, ".join(l)
print(result)
```

The above line will return "apple, and, mango, and, banana".

→ Format Method (strings)

formats the values inside the string into a desired output.

```
template.format(p1, p2...)
```

Syntax:

```
"{} is a good {}".format("harry","boy")    #1.
"{1} is a good {0}".format("harry","boy")   #2.

#output for 1.
#harry is a good boy

#output for 2.
#boy is a good harry
```

→ Map, Filter & Reduce

Map applies a function to all the items in an input_list.

Syntax:

```
map(function, input_list)
#                  #the function can be lambda
                    function
```

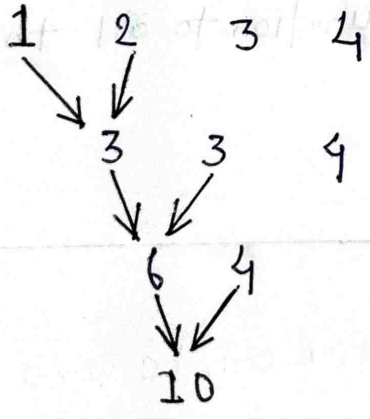
Filters creates a list of items for which the function returns true.

```
list(filter(function))
                    # the function can be lambda function
```

Reduce applies a rolling computation to sequential pair of elements.

```
from functools import reduce
val = reduce (function, List1)
                    # the function can be lambda
                      function
```

If the function compute sum of two numbers & the list is [1,2,3,4]



sequential computation

Practice set

1. Create two virtual enviroments, install few packages in the first one. How do you create a similiar enviroment in the second one?

2. Write a program to input name, marks & phone number of a student & format it using the format function like below:

"The name of the student is Harry, his marks are 72 & phone number is 99999888"

3. A list contains the Multiplication table of 7. Write a program to convert it to vertical string of same numbers.

To activat virtual enviroment

1. write name like - env1
2. Tab → the this appears .\env1\
3. write scripts → .\env1\scripts\
4. write activate.ps1 → .\env1\scripts\activate.ps1

④ Write a program to filter a list of numbers which are divisible by 5.

⑤ Write a program to find the maximum of the numbers in a list using the reduce function.

⑥ Run pip freeze for the system interpreter. Take the contents & create similiar virtualenv.

⑦ Explore the 'Flask' module & create a web server using Flask & python.